

# Teaching the Machine Learning Story

*A hands-on workshop connecting critical data literacy  
and explainable machine learning, for educators who teach statistics.*

Every number, plot, and exercise in this manual comes from actually running `icarm` v0.1.0 on `student-mat.csv` (395 students, seed = 2025). Nothing is simulated.

## Contents

---

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| <b>1</b>  | <b>How to Use This Guide</b>                                 | <b>2</b>  |
| 1.1       | What You Need Before You Start . . . . .                     | 2         |
| <b>2</b>  | <b>Learning Objectives</b>                                   | <b>3</b>  |
| <b>3</b>  | <b>This Workshop Follows a Story Arc</b>                     | <b>4</b>  |
| <b>4</b>  | <b>Critical Data Literacy + Explainable Machine Learning</b> | <b>5</b>  |
| <b>5</b>  | <b>The Four C's: A Framework for Honest Stories</b>          | <b>6</b>  |
| <b>6</b>  | <b>The Machine Learning Pipeline: A Map</b>                  | <b>7</b>  |
| <b>7</b>  | <b>Stage 1: Ask</b>                                          | <b>8</b>  |
| <b>8</b>  | <b>Stage 2: Data</b>                                         | <b>9</b>  |
| <b>9</b>  | <b>Stage 3: Split</b>                                        | <b>10</b> |
| <b>10</b> | <b>Stage 4: Train</b>                                        | <b>12</b> |
| <b>11</b> | <b>Stage 5: Evaluate</b>                                     | <b>14</b> |
| <b>12</b> | <b>Stage 6: Explain</b>                                      | <b>16</b> |
| <b>13</b> | <b>Stage 7: Audit</b>                                        | <b>18</b> |
| <b>14</b> | <b>Stage 8: Decide</b>                                       | <b>21</b> |
| <b>15</b> | <b>Common Storytelling Mistakes</b>                          | <b>22</b> |
| <b>16</b> | <b>Capstone Assignment: Solve Your Own Case</b>              | <b>23</b> |
| <b>17</b> | <b>From Workshop to Classroom</b>                            | <b>25</b> |
| <b>A</b>  | <b>Full icarm Function Glossary</b>                          | <b>26</b> |
| <b>B</b>  | <b>Consolidated Code Booklet</b>                             | <b>27</b> |
| <b>C</b>  | <b>References and Further Reading</b>                        | <b>29</b> |

# 1 How to Use This Guide

This manual combines three things that are normally handed out as separate documents:

- **The lecture** — concept explanations for every stage of a machine learning pipeline, written for an audience of statistics educators with no assumed ML background.
- **The lab** — comprehensive, runnable R code using the `icarm` package, organised into instructor-led demonstrations and participant exercises.
- **The workshop** — timing, activities, discussion questions, and a capstone assignment, so this document alone is enough to run the session.

**A note on honesty.** Every statistic, tree split, coefficient, and fairness gap in this manual was produced by actually running the code shown, on the real dataset, once, with a fixed random seed (2025). Where a number could plausibly change with a different seed, that is said explicitly. Nothing here is invented for effect.

## 1.1 What You Need Before You Start

Listing 1: Setup — run this before the workshop begins

```
1 install.packages("icarm")
2 install.packages(c("randomForest", "e1071", "glmnet"))
3 # or, if installing from a local source tarball:
4 # install.packages("icarm_0.1.0.tar.gz", repos = NULL, type = "source")
5
6 library(icarm)
7 library(dplyr)
8 library(ggplot2)
```

Download `student-mat.csv` from the UCI Machine Learning Repository (Cortez & Silva, 2008) and place it in your working directory. Every exercise in this manual assumes it is loaded as follows:

Listing 2: Loading the dataset used throughout this workshop

```
1 students <- read.csv2("student-mat.csv", stringsAsFactors = TRUE)
2 students$pass <- factor(ifelse(students$G3 >= 10, "Pass", "Fail"))
3 dim(students)      # 395 students, 33 attributes + 1 target
4 table(students$pass)
```

## 2 Learning Objectives

---

By the end of this workshop, participants will be able to:

1. Explain the eight-stage machine learning pipeline — **Ask, Data, Split, Train, Evaluate, Explain, Audit, Decide** — well enough to teach it.
2. Fit, compare, and evaluate five different model types in R using one consistent interface (`icarm_fit()`).
3. Explain *how* a decision tree, a logistic regression, and a random forest each actually make a prediction, mechanically.
4. Apply a simple four-part framework — **Claim, Context, Case, Caveat** — to turn any statistical result into an honest, well-told story.
5. Run a group fairness audit, and correctly distinguish a genuine disparity from statistical noise.
6. Articulate, in one sentence, how critical data literacy and explainable machine learning are two views of the same underlying practice.
7. Leave with a specific, low-prep classroom activity for at least four stages of the pipeline, and a complete capstone assignment they can assign to their own students.

### 3 This Workshop Follows a Story Arc

A machine learning pipeline is usually taught as a checklist. It is more useful — and much easier to remember — taught as a story, because it already has a story’s shape: a setup, a rising complication, a turning point, a climax, and a resolution. Table 1 maps the whole workshop onto that shape; we will refer back to it throughout the day.

| Story beat    | Pipeline stage(s) | What happens                                                                         |
|---------------|-------------------|--------------------------------------------------------------------------------------|
| Setup         | Ask + Data        | What are we trying to find out, and what do we actually have to work with?           |
| Rising action | Split + Train     | Build the tools honestly — hold out real evidence, try more than one model.          |
| Complication  | Evaluate          | The first model looks fine, until one metric (specificity) says otherwise.           |
| Turning point | Explain           | The model’s strongest, most confident rule turns out to be wrong for a real student. |
| Climax        | Audit             | A real, sizeable fairness gap appears — the moment the stakes become concrete.       |
| Resolution    | Decide            | Document everything, so the finding survives past this room.                         |

Table 1: The story arc this workshop follows, mapped onto the ML pipeline.

## 4 Critical Data Literacy + Explainable Machine Learning

This is the thesis that connects everything in this manual, worth stating plainly before we begin:

**Critical data literacy** interrogates the *data going in* — what is measured, what is a proxy for something else, whose story is missing from the columns we were handed. **Explainable machine learning** interrogates the *decisions coming out* — why the model said what it said, and who it fails. Neither alone is enough. A dataset can be criticised forever without ever pinning down what a specific model actually does with it; a model can be explained in exhaustive technical detail while missing that one of its inputs is a proxy for something structural. This workshop is one continuous argument, not two adjacent topics.

Table 2 makes the connection concrete, stage by stage.

| Stage   | Critical data literacy question                                                                 | What explainable ML gives you                                                                                |
|---------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Data    | What does this variable actually measure? Whose story is missing?                               | Nothing yet — this is upstream of any model, and that is itself worth teaching.                              |
| Train   | Does a black box, chosen for convenience, trade away anyone's ability to hold this accountable? | The interpretable-vs-extended choice, made visible instead of implicit.                                      |
| Explain | When a variable matters most, whose social reality does it actually encode?                     | Global importance and local decision paths — “this matters” becomes “here is exactly how, for this student.” |
| Audit   | Who is failed by this system, and is that failure evenly spread?                                | Fairness metrics turn a suspicion into a number: a 10% vs. 39% catch-rate.                                   |
| Decide  | Who is accountable if this is wrong, and how would anyone find out?                             | The audit trail / scorecard — a documented answer, not just a checked box.                                   |

Table 2: Critical data literacy and explainable ML, mapped onto the same pipeline.

We will return to this table explicitly at Stage 7 (Audit), where a variable flagged as a proxy back in Stage 2 (Data) reappears as a measured, numerical fairness gap — the moment critical literacy and explainability converge on the same finding.

## 5 The Four C's: A Framework for Honest Stories

---

A number without a story invites the listener to invent one — usually the most dramatic version, not the most accurate one. “68% accuracy” means nothing on its own; the person hearing it will decide what it means anyway, so the storyteller’s job is to make sure they decide based on the right context. Use this four-part structure every time a result is presented, in this order, every time:

- 
- |   |                |                                                                                                              |
|---|----------------|--------------------------------------------------------------------------------------------------------------|
| ➊ | <b>Claim</b>   | State the finding precisely. The exact number, not a rounded impression: “63% accuracy,” not “pretty good.”  |
| ➋ | <b>Context</b> | Translate the number into what it means for a real decision. What does 63% actually let you do, or not do?   |
| ➌ | <b>Case</b>    | Ground it in ONE concrete, specific example — a real row, a real case — not just an aggregate statistic.     |
| ➍ | <b>Caveat</b>  | Name a limitation, an alternative explanation, or a small sample size. Every honest result has at least one. |
- 

Grade student write-ups with a simple rubric tied directly to these four parts: Claim correct (25%), Context relevant to a real decision (25%), Case specific and real, not invented (25%), Caveat genuine and not an afterthought (25%). This rubric appears again, fully worked out, in the Capstone Assignment (Section 16).

## 6 The Machine Learning Pipeline: A Map

---

Every machine learning project, however complicated it looks from the outside, is a walk through the same eight stages. This workshop walks it once, slowly, in order, using one real dataset throughout.



**Ask** — what question, and what kind (yes/no? a number? a category?)

**Data** — what's actually in the rows, and what isn't

**Split** — hold out data the model has never seen, to test honestly

**Train** — fit one or more models to the training data

**Evaluate** — score the model on the held-out data

**Explain** — understand *why* the model predicts what it predicts

**Audit** — check fairness, calibration, and decision thresholds

**Decide** — document the result and decide whether/how to use it

The rest of this manual walks these eight stages in order. Each stage includes: a plain-language concept explanation, a real worked example told through the Four C's, one or more hands-on coding exercises, a classroom adaptation idea, and discussion questions.



## 7 Stage 1: Ask

Every model starts with a target variable and a task type. Here: `pass = 1` if the final grade `G3`  $\geq 10$ , else 0. `icarm` auto-detects that two outcome levels means a **binary classification** task — the same 395 students could just as easily support a **regression** task (predict the exact grade) or a **multiclass** task (predict a performance tier). The question you ask shapes everything downstream; it is not a neutral first step.

Before any modeling, have students write the target variable and task type as a plain sentence: “We are predicting whether a student passes (yes/no), using everything else we know about them.” Grade the sentence, not the code — most modeling mistakes are really question-framing mistakes in disguise.

### Exercise 1: Naming the Question (5 minutes, pairs, no computer)

With your partner, write down, in one sentence each:

1. What are we predicting, exactly?
2. Is that a yes/no question, a number, or a category with more than two options?
3. Who would use this prediction, and for what decision?

Compare your answer to question 3 with the pair next to you. It is normal, and useful, if they differ.

## 8 Stage 2: Data

395 students, 33 attributes, two real schools (Gabriel Pereira and Mousinho da Silveira). Before fitting anything: which variables are *hard facts* (age, school), which are *self-reported perceptions* (family relationship quality, health, all on 1–5 scales), which are *proxies* for something else (address as a proxy for resource access), and which are *sensitive* (two columns record alcohol use for 15–18 year-olds)? None of this requires code — it requires reading the codebook critically, before a single model exists.

**REAL RESULT** The dataset at a glance: 395 students · 33 attributes · 2 schools · target pass = 265 Pass / 130 Fail.

### Exercise 2: Sort the Evidence (15 minutes, pairs, no computer)

Using the attribute list handed out separately (or the codebook comments in the setup code), sort all 33 variables into four bins:

| Category    | Definition                                        | Examples                 |
|-------------|---------------------------------------------------|--------------------------|
| Hard fact   | Directly observable, low ambiguity                | age, school, sex         |
| Self-report | A 1–5 perception, not a measurement               | famrel, health, freetime |
| Proxy       | Stands in for something not directly measured     | address, Medu, Fedu      |
| Sensitive   | Personal; deserves scrutiny in a teaching dataset | Dalc, Walc, romantic     |

**Debrief question:** which single variable was hardest to place, and why? Disagreement here is the point — it means the class is actually thinking about what a column represents, not just what it's named.

**Chief teaching note.** failures will dominate almost every model built today. Ask the room, before revealing any model output: is that insight, or is the model just restating the past?

### Discussion questions.

- Why might a dataset used in thousands of ML tutorials worldwide casually include two columns on minors' alcohol use?
- What's missing from these 33 columns that would matter for understanding student outcomes — teacher quality? school funding? mental health? home language?
- If you had to remove five variables before anyone could use this dataset, which five, and why?

## 9 Stage 3: Split

A model that is only ever tested on the data it was trained on will always look better than it really is. `icarm_split()` holds out 25% of students (100 of 395) that the model never sees during training — the only honest way to estimate how it will perform on a *new* student. This is the single most important idea in the entire pipeline, and everything that follows depends on taking it seriously.

### Exercise 3: Splitting Honestly

```
1 sp      <- icarm_split(students, prop = 0.75, seed = 2025, stratify = "pass")
2 train   <- sp$train
3 test    <- sp$test
4
5 # Drop G1/G2 (prior-period grades). They are near-perfect predictors of G3
6 # and would hide the more interesting behavioural variables we want to
7 # reason about later.
8 train_bin <- train %>% select(-G1, -G2, -G3)
9 test_bin  <- test  %>% select(-G1, -G2, -G3)
10
11 nrow(train_bin) # TODO: how many students is this?
12 nrow(test_bin)  # TODO: how many students is this?
```

Fill in the two TODO comments by running the code, before reading further.

### Exercise 4: The Generalization Gap (10 minutes, coded)

```
1 m_cart <- icarm_fit(pass ~ ., train_bin, model = "cart", positive = "Pass")
2
3 train_pred <- predict(m_cart, train_bin, type = "class")
4 test_pred  <- predict(m_cart, test_bin,  type = "class")
5
6 icarm_metrics(train_bin$pass, train_pred, positive = "Pass")["accuracy"]
7 icarm_metrics(test_bin$pass,  test_pred,  positive = "Pass")["accuracy"]
```

**Before running this**, predict with your partner: will the training accuracy be higher, lower, or about the same as the test accuracy? By how much?

**REAL RESULT** The real gap: this exact decision tree scores **0.78 accuracy on the data it was trained on**, and **0.63 accuracy on students it has never seen**. A 15-point drop, from the same model, on the same underlying pattern.

### THE STORY — THE FOUR C'S

**Claim:** the same decision tree scores 15 points higher on training data (0.78) than on held-out data (0.63).

**Context:** a model's performance on its own training data tells you almost nothing about how it will do on anyone new.

**Case:** every number in the rest of this workshop is a held-out, test-set number, for exactly this reason.

**Caveat:** this specific 15-point gap is one split, one seed; the direction (train > test) is the reliable part, not the exact size.

Fit the same model twice in front of the class: once scored only on training data, once on held-out data. Ask students to commit to a prediction, in writing, *before* seeing either number. Public wrong predictions about overfitting are more memorable than a correct lecture about it.

**Discussion questions.**

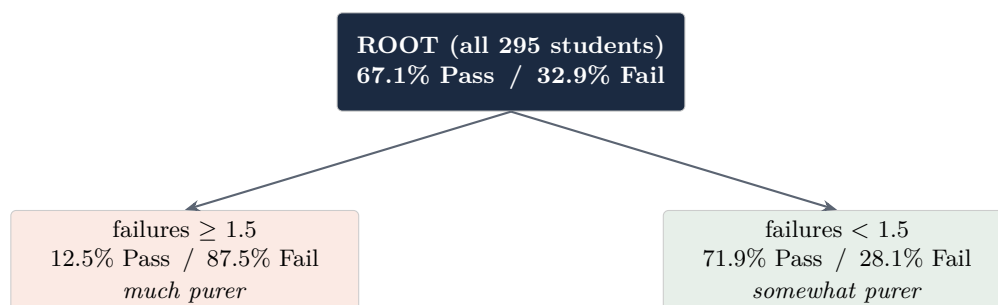
- If a colleague showed you only the training accuracy of their model, what one question would you ask before believing it?
- Why does `stratify = "pass"` matter when splitting? What could go wrong without it?

## 10 Stage 4: Train

`icarm_fit()` fits *interpretable* models — decision trees, logistic regression, sparse logistic regression — and *extended* (“black-box”) models — random forest, SVM — through the same interface. The choice is not just about accuracy: an interpretable model lets you trace any single prediction back to specific, human-readable rules.

### How a Decision Tree Actually Decides

A tree repeatedly asks the yes/no question that best separates Pass from Fail, measured by how “mixed” each resulting group still is.



**REAL RESULT** The tree tries every possible yes/no question on every variable, picks the one that makes the two resulting groups as one-sided as possible, then repeats the process separately inside each new group. That is the entire algorithm — there is no additional machinery.

### How Logistic Regression Weighs Evidence

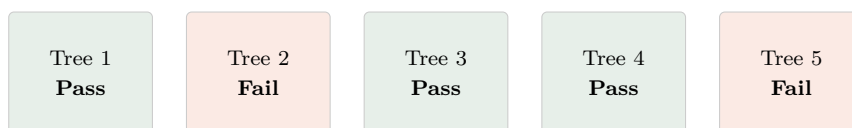
Every variable gets a signed weight. Multiply each student’s value by its weight, add everything up, then squeeze the total through a curve that always outputs a number between 0 and 1.

- |   |                                   |                                                                                                                                   |
|---|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| ❶ | <b>Weigh each variable</b>        | The model learns one number (a weight) per variable from training data — positive weights push toward Pass, negative toward Fail. |
| ❷ | <b>Add them up</b>                | For one student, multiply their value on each variable by its weight, and sum everything into a single number.                    |
| ❸ | <b>Squeeze into a probability</b> | That sum is fed through the logistic curve, mapping any number into a probability between 0 and 1.                                |

**REAL RESULT** Real weight from this model: each additional prior failure lowers a student’s log-odds of passing by about 1.24 — statistically significant ( $p < 0.001$ ). Negative weight, exactly as the decision tree’s first split already suggested.

### How a Random Forest Votes

Many trees, each seeing a different random slice of the data and variables, vote; the majority wins.



3 of 5 vote Pass  $\Rightarrow$  forest predicts Pass, confidence 60%

**REAL RESULT** The real random forest in this workshop uses hundreds of trees, not five, but the mechanism is exactly the same. That is also why it is a black box: there is no single path to point to, only a vote count across hundreds of different trees.

### Exercise 5: Fit Your First Model

```
1 m_cart <- icarm_fit(pass ~ ., train_bin, model = "cart", positive = "Pass")
2 print(m_cart)           # icarm's own model card
3 print(m_cart$fit)       # the full rpart tree text -- find the root split
```

Find the first (root) split in the printed tree. Does it match the variable you and your partner guessed back in the icebreaker discussion?

### Exercise 6: Fit Four More Models (assignment: complete the blanks)

```
1 m_logit <- icarm_fit(pass ~ ., train_bin, model = "_____", positive = "
  Pass")
2 m_l1    <- icarm_fit(pass ~ ., train_bin, model = "logistic_l1", positive =
  "Pass")
3 m_rf    <- icarm_fit(pass ~ ., train_bin, model = "_____", positive = "
  Pass")
4 m_svm   <- icarm_fit(pass ~ ., train_bin, model = "svm", positive =
  "Pass")
```

Fill in the two blanks using the model names introduced in this section. Then confirm all four models fitted without error before moving to Stage 5.

Split the class in half. One half fits an interpretable model, the other a black-box model, on the same data. Debrief with one question: “can you explain ONE student’s prediction to the person next to you, right now, without a computer?” Only one half of the room will be able to say yes.

## 11 Stage 5: Evaluate

A single “accuracy” number hides *how* a model is wrong. `icarm_metrics()` and `icarm_compare()` report accuracy, precision, recall, specificity, and F1 together — and different tasks (binary, multiclass, regression) get different, appropriate metrics automatically.

### Exercise 7: Confusion Matrix by Hand (5 minutes, no computer)

A small model was tested on 20 held-out students. Its confusion matrix is:

|                | Actual Pass | Actual Fail |
|----------------|-------------|-------------|
| Predicted Pass | 11          | 4           |
| Predicted Fail | 2           | 3           |

By hand: compute accuracy, precision (for Pass), and recall (for Pass). Compare your answers with your partner before checking with `icarm_metrics()` in the next exercise.

### Exercise 8: The Real Numbers

```
1 pred_cart <- predict(m_cart, test_bin, type = "class")
2 icarm_metrics(test_bin$pass, pred_cart, positive = "Pass")
3 icarm_plot_confusion(test_bin$pass, pred_cart, title = "CART: Predicting Pass
  /Fail")
```

**REAL RESULT** accuracy 0.63   balanced\_acc 0.53   f1 0.75   precision 0.69   recall 0.82   specificity 0.24

### THE STORY — THE FOUR C'S

**Claim:** the tree is 63% accurate overall, but its specificity is only 0.24.

**Context:** specificity of 0.24 means of every 4 students who actually fail, this model correctly flags only about 1.

**Case:** the tree's first question is simply “has this student failed before?” — students who have are predicted Fail 87.5% of the time.

**Caveat:** this model saw no prior-period grades (G1/G2), which are much stronger predictors of G3; a model using them would likely score higher on accuracy alone.

Listing 3: Compare all five models at once

```
1 cmp <- icarm_compare(
2   list(CART = m_cart, Logistic = m_logit, 'Logistic (L1)' = m_l1,
3     RandomForest = m_rf, SVM = m_svm),
4   test_data = test_bin, outcome = "pass", positive = "Pass"
5 )
6 print(cmp)
7 icarm_plot_comparison(cmp, metrics = c("accuracy", "f1"))
```

**REAL RESULT** Accuracy: CART 0.63 · Logistic 0.65 · **Logistic (L1) 0.70 (best)** · Random Forest 0.68 · SVM 0.69.

**THE STORY — THE FOUR C'S**

**Claim:** sparse (L1) logistic regression scored highest of five models — 0.70 accuracy — ahead of random forest (0.68) and SVM (0.69).

**Context:** the best-performing model here is also the most explainable one; there was no accuracy reason to reach for a black box.

**Case:** for any student, the logistic model's prediction traces to specific weighted factors, no post-hoc tool required.

**Caveat:** this ranking reflects one split (seed 2025); a different split could reorder the middle, though the top result was stable in testing.

Hand out a small, filled-in confusion matrix on paper (as in the exercise above) before ever showing `icarm_metrics()` computing it automatically. Students who compute accuracy and recall by hand once never again confuse the two.

**Discussion questions.**

- Is a 2-point accuracy gap (0.68 to 0.70) worth the loss of interpretability that would come from picking a black box instead? Defend either answer.
- Why might balanced accuracy (0.53) tell a more honest story than plain accuracy (0.63) here?



## 12 Stage 6: Explain

Two different questions need two different tools. **Global** explanation asks “what does this model generally pay attention to?” **Local** explanation asks “why did it make *this* prediction, for *this* student?” A pattern that holds on average is not automatically true of any one individual.

### Part A — Global Explanation

#### Exercise 9: What Does the Model Generally Look At?

```
1 ex_rf <- icarm_explain(m_rf, data = test_bin, label = "Random Forest")
2 ex_rf$importance %>% arrange(desc(importance)) %>% head(6)
3 icarm_plot_importance(ex_rf, n_features = 12, title = "Random Forest: global
  importance")
4
5 ex_cart <- icarm_explain(m_cart, data = test_bin, label = "CART")
6 ex_cart$importance %>% arrange(desc(importance)) %>% head(6)
```

**REAL RESULT** Random Forest top 6: failures (12.4), absences (10.7), goout (8.0), Mjob (7.5), age (7.1), reason (5.7).

**CART top 6:** failures (15.6), goout (4.9), guardian (3.7), age (3.0), Fedu (2.9), reason (2.8).

#### THE STORY — THE FOUR C’S

**Claim:** both models rank prior failures as the top predictor, by a wide margin.

**Context:** agreement between a transparent model and a black box is reassuring — not one algorithm’s quirk.

**Case:** CART’s importance for **failures** (15.6) is over 3× its next variable; the forest shows the same shape.

**Caveat:** the two models disagree on what’s #2 — **absences** for the forest, **goout** for the tree. “Importance” depends partly on the model, not only on the data.

### Part B — Local Explanation: Three Real Cases

#### Exercise 10: Meet Three Real Students

```
1 case_a_id <- which(test_bin$pass == "Pass" & pred_cart == "Pass")[1] # row
  3
2 case_b_id <- 28 # row
  28
3 case_c_id <- which(pred_cart == "Fail" & test_bin$pass == "Pass")[1] # row
  2
4
5 probs <- predict(m_cart, test_bin, type = "prob")
6 probs[c(case_a_id, case_b_id, case_c_id), ]
```

|                   | Student A (row 3) | Student B (row 28) | Student C (row 2) |
|-------------------|-------------------|--------------------|-------------------|
| failures          | 0                 | 1                  | <b>3</b>          |
| goout             | 2                 | 4                  | 2                 |
| Model predicts    | Pass, 78.8%       | Fail, 64.5%        | Fail, 87.5%       |
| Actually happened | Pass              | Fail               | <b>Passed</b>     |
| Verdict           | Correct           | Correct            | <b>Wrong</b>      |

### THE STORY — THE FOUR C'S

**Claim:** for Student C, the model's single strongest rule (3+ prior failures) predicted Fail with 87.5% confidence.

**Context:** that rule is the most reliable-looking pattern in the entire dataset; confidence this high is rare.

**Case:** this specific, real student passed anyway.

**Caveat:** one row out of 100 doesn't overturn the pattern — but it shows a strong aggregate pattern is never a guarantee for any one person.

### Exercise 11: Write the Case Note (15 minutes, pairs)

Your pair is assigned **one** of the three students above. Write a five-sentence case note:

1. What did the model predict, and how confident was it?
2. Name the 2–3 factors behind that prediction.
3. Reframe *one* factor in human terms (e.g. “low weekly study time” could mean a job, caregiving, or no quiet place to study).
4. Name one thing the model cannot see about this student.
5. Would you hand this case note to a real teacher? Why or why not?

Groups assigned Student C should read theirs aloud last.

The “five-sentence case note” is a complete, gradable writing assignment on its own — it needs no computer once the local prediction and its top factors are handed to students on paper. Use a simple rubric: one point per sentence, reduced by half if sentence 3 fails to reframe the factor in genuinely human terms rather than restating the statistic.

### Discussion questions.

- Student C's prediction used the model's single most trusted rule. Does that make the error more or less concerning than an error on an uncertain prediction?
- **failures** dominates almost every local explanation today. Is the model explaining Student C's situation, or mostly restating that Student C has struggled before? What is the difference?

## 13 Stage 7: Audit

A model can score well overall while treating groups very differently — and a model's stated probabilities can be systematically over- or under-confident. `icarm_fairness()` checks the first; `icarm_calibrate()` and `icarm_thresholds()` check the second.

**Callback to Stage 2.** We flagged `address` as a proxy for resource access before any model existed. This is what that suspicion looks like once you can actually measure it: critical data literacy and explainable machine learning converging on the same finding.

### The Headline Finding

#### Exercise 12: Auditing by Sex

```
1 fair_sex <- icarm_fairness(m_rf, test_bin, outcome = "pass",
2                           protected = "sex", positive = "Pass")
3 print(fair_sex)
4 icarm_plot_fairness(fair_sex, metric = "acc", title = "Accuracy by sex")
5 icarm_equity_summary(fair_sex)
```

#### REAL RESULT

| Group  | n  | Accuracy | Actual Fails caught |
|--------|----|----------|---------------------|
| Female | 56 | 0.625    | 2 of 20 (10%)       |
| Male   | 44 | 0.750    | 5 of 13 (38.5%)     |

#### THE STORY — THE FOUR C'S

**Claim:** the model correctly identifies 10% of actually-failing female students, versus 39% of actually-failing male students.

**Context:** if this triggered a support intervention, nine of ten female students who need it would never be flagged.

**Case:** in raw counts, 2 of 20 actually-failing female students were caught, vs. 5 of 13 male — both real, sizeable subgroups, not statistical noise.

**Caveat:** this is from one 100-student test split; the gap's existence is worth investigating further before drawing conclusions about cause.

### When “Equal” Accuracy Isn’t Equal Treatment

#### Exercise 13: Auditing by Address

```
1 fair_addr <- icarm_fairness(m_rf, test_bin, outcome = "pass",
2                           protected = "address", positive = "Pass")
3 print(fair_addr)
4 icarm_plot_fairness(fair_addr, metric = "acc", title = "Accuracy: urban vs.
5                     rural")
```

**REAL RESULT** Rural ( $n = 25$ ): accuracy **0.68**, catches 3 of 10 actual Fails (30%). Urban ( $n = 75$ ): accuracy **0.68**, catches 4 of 23 actual Fails (17%).

### THE STORY — THE FOUR C'S

**Claim:** accuracy is identical — 0.68 for both rural and urban students.

**Context:** stopping at overall accuracy would lead you to conclude there is no fairness issue at all between these two groups.

**Case:** the rate of correctly catching actual failing students differs — 30% rural vs. 17% urban — a real difference in the type of error made.

**Caveat:** both catch-rates are low in absolute terms; this model is weak at its actual job for everyone, and the group comparison shouldn't distract from that bigger problem.

## Can You Trust the Probability?

### Exercise 14: Calibration and Thresholds

```
1 probs <- predict(m_rf, test_bin, type = "prob")[, "Pass"]
2
3 cal <- icarm_calibrate(m_rf, test_bin, outcome = "pass", positive = "Pass")
4 print(cal)
5 icarm_plot_calibration(cal, title = "Is a 0.7 predicted probability really
6   70%?")
7
8 thr <- icarm_thresholds(test_bin$pass, probs, positive = "Pass")
9 print(thr, n = 20)
10 icarm_plot_thresholds(thr, metrics = c("accuracy", "recall", "precision", "f1"))
```

**REAL RESULT** Brier score **0.197**, Expected Calibration Error **0.124** — rated **“POOR”** by `icarm`'s own calibration check. Raising the threshold from 0.50 to 0.60 improves accuracy from 0.68 to 0.75.

### Exercise 15: Set the Threshold (role play, 10 minutes)

Half the room plays **Role A: school counsellor** (missing a struggling student is the costly mistake). The other half plays **Role B: scholarship committee** (wrongly awarding a scholarship is the costly mistake). Each role picks one row from the threshold table and defends it in one sentence. Same model, same numbers, two different, defensible decisions.

Fairness audits need no coding to discuss: hand students a pre-computed two-group table (as in the boxes above) and ask them to argue for measurement bias, structural bias, or model bias — a full class period without opening R.

### Discussion questions.

- Propose one intervention for the sex gap that is *not* “just delete the `sex` column.” Why might deleting it fail to fix anything?

- A model with an ECE of 0.124 is called “poorly calibrated.” What would you tell a school administrator who wants to use its probabilities directly to rank students?

## 14 Stage 8: Decide

A model's numbers should be documented, not just discoverable on demand. `icarm_audit()` and `icarm_scorecard()` produce a timestamped, reproducible record of what was fit, how it performed, and how fair it was — the artifact that should travel with a model if it is ever actually used, separate from any slide deck or lecture.

### Exercise 16: Generate the Audit Trail

```
1 metrics_rf <- icarm_metrics(test_bin$pass, predict(m_rf, test_bin, type = "
  class"),
2                               y_prob = probs, positive = "Pass")
3
4 icarm_audit(m_rf, metrics = metrics_rf, fairness = fair_sex,
5             notes = "ICOTS workshop model; not for real deployment decisions.",
6             analyst = "Your Name", path = "rf_audit_trail.json")
7
8 icarm_scorecard(m_rf, test_bin, outcome = "pass", protected = "sex",
9                 positive = "Pass", analyst = "Your Name",
10                project = "ICOTS Workshop", path = "rf_scorecard.json")
```

**REAL RESULT** Real JSON output (excerpt): "accuracy": 0.68, "balanced\_acc": 0.5613, "max\_acc\_gap": 0.125, "max\_eo\_gap": 0.2846, "disparate\_impact\_pass": true.

### THE STORY — THE FOUR C'S

**Claim:** `icarm_audit()` and `icarm_scorecard()` produced a timestamped, reproducible record of this model's performance and fairness numbers.

**Context:** the claims made across Stages 5–7 are not just slide text — they're reproducible from a file anyone can re-check.

**Case:** the scorecard, not a slide deck, is what should travel with a model if it's ever deployed.

**Caveat:** a scorecard documents what was measured; it does not decide whether the numbers are good enough to deploy on — that judgment still needs a person.

Assign a one-paragraph “deployment memo” as a capstone: given a model's scorecard, should it be used, used with safeguards, or not used at all? Grade the reasoning, not the recommendation — there is no single correct answer.

## 15 Common Storytelling Mistakes

---

| Mistake                              | Why it misleads                                                                                                    |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Skipping the Caveat                  | Claim + Context alone makes a result sound more solid than it is. Every result has a limit — find it, say it.      |
| Treating an aggregate as a guarantee | “82% of students with these traits pass” does not mean any specific student will. Say so explicitly.               |
| Cherry-picking the example case      | If every case shown is a success, the story is advertising, not analysis. Include a case where the model is wrong. |
| Reporting one metric only            | Equal accuracy across groups can hide a real gap in which errors are made. Check more than one number.             |
| Trusting a small subgroup percentage | A 90% catch-rate on 5 people is one or two individuals. Report the count, not just the percentage.                 |

## 16 Capstone Assignment: Solve Your Own Case

### Overview

Working in your assigned pair, you will repeat Stages 3–7 of this workshop using a **different protected attribute**, and produce a short written deliverable using the Four C's. This assignment can be completed in the final 15 minutes of the workshop (abbreviated) or assigned as take-home work (full version, recommended).

### Step 1 — Choose Your Attribute

Each pair chooses one attribute, not already used in the main workshop, from Table 3. No two pairs in the room should choose the same one.

| Attribute | What it represents                                       |
|-----------|----------------------------------------------------------|
| Pstatus   | Parents' cohabitation status (living together vs. apart) |
| internet  | Internet access at home (yes/no)                         |
| schoolsup | Extra educational support from the school (yes/no)       |
| guardian  | Student's guardian (mother, father, or other)            |

Table 3: Capstone attribute choices.

### Step 2 — Run the Pipeline

Listing 4: Capstone template — change ATTR and run

```

1 ATTR <- "Pstatus" # replace with your assigned attribute
2
3 # Stage 5: evaluate the model you already have (m_rf) on your attribute
4 team_fair <- icarm_fairness(m_rf, test_bin, outcome = "pass",
5                           protected = ATTR, positive = "Pass")
6 print(team_fair)
7 icarm_plot_fairness(team_fair, metric = "acc",
8                     title = paste("Accuracy by", ATTR))
9 icarm_equity_summary(team_fair)
10
11 # Stage 6: pick one real case from your test set and describe its local path
12 # (reuse the CART tree text from Stage 4 -- find where your case's row lands)
13 your_case_id <- 1 # TODO: pick a specific test_bin row number
14 test_bin[your_case_id, c(ATTR, "failures", "goout", "pass")]
15 predict(m_cart, test_bin[your_case_id, ], type = "prob")

```

### Step 3 — Write the Story

Produce a written deliverable, no more than one page, containing:

1. **One plain-language sentence** stating what the model predicts and how well, for your attribute's groups.
2. **One local case story**, written as a five-sentence case note (Stage 6 format).
3. **One fairness finding**, applying the Four C's in full (Claim, Context, Case, Caveat).
4. **One recommendation**: deploy as-is, deploy with safeguards, or do not deploy — with a one-sentence justification.



## Grading Rubric

| Component<br>(25% each) | What earns full credit                                                                                                       |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Claim                   | The exact number is stated correctly, with no rounding that changes its meaning.                                             |
| Context                 | The number is connected to a real, specific decision or consequence — not left abstract.                                     |
| Case                    | A real row from the test set is used, not an invented or composite example.                                                  |
| Caveat                  | A genuine limitation is named — small sample size, single split, or an alternative explanation — not a throwaway disclaimer. |

**For facilitators running this as take-home work:** collect the ATTR choice from each pair at the end of the workshop to guarantee no duplicates, and set a firm one-page limit. Pairs consistently write more convincing Case sections when required to quote the exact row values, as in the Step 2 template.

## 17 From Workshop to Classroom

---

Every dataset used here is public and free (UCI Machine Learning Repository). Substitute any dataset your own students already know — the pipeline and the Four C's do not change.

| Format                         | Suggested adaptation                                                                                                             |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| A single 75-minute lecture     | Show Stage 3 (Split) and one worked example (the Student C surprise, Stage 6). Skip coding entirely — present the results as-is. |
| A one-week module              | Students run the Code Booklet themselves through Stage 5 (Evaluate), then write one Four C's paragraph as homework.              |
| A full course unit (2–3 weeks) | Add Stages 6–8. Assign the Capstone (Section 16) as a graded deliverable.                                                        |
| Assessment, any format         | Use the Capstone rubric directly: Claim, Context, Case, Caveat, 25% each.                                                        |

## A Full icarm Function Glossary

| Function                                                                                                   | Stage   | Verified use in this workshop                                     |
|------------------------------------------------------------------------------------------------------------|---------|-------------------------------------------------------------------|
| <code>icarm_split()</code>                                                                                 | 3       | Stratified 75/25 split, seed 2025                                 |
| <code>icarm_fit()</code>                                                                                   | 4, 8    | "cart", "logistic", "logistic_l1",<br>"random_forest", "svm"      |
| <code>predict.icarm_model()</code>                                                                         | 4, 6, 7 | Class, probability, and regression predictions                    |
| <code>print()</code> / <code>summary()</code>                                                              | 4       | Inspecting the fitted CART tree                                   |
| <code>icarm_metrics()</code>                                                                               | 5, 8    | Binary, regression, and multiclass metrics                        |
| <code>icarm_plot_confusion()</code>                                                                        | 5       | Binary and 3-class confusion matrices                             |
| <code>icarm_compare()</code><br><code>icarm_plot_comparison()</code>                                       | 5       | 5-model side-by-side comparison                                   |
| <code>icarm_explain()</code><br><code>icarm_plot_importance()</code>                                       | 6       | Global importance for CART and random forest                      |
| <code>icarm_explain_local()</code>                                                                         | 6       | Requires the optional DALEX package; not required for this manual |
| <code>icarm_fairness()</code><br><code>icarm_plot_fairness()</code><br><code>icarm_equity_summary()</code> | 7       | <b>sex</b> and <b>address</b> audits                              |
| <code>icarm_equalized_odds_curve()</code><br><code>icarm_plot_roc_groups()</code>                          | 7       | Per-group ROC across thresholds                                   |
| <code>icarm_calibrate()</code><br><code>icarm_plot_calibration()</code>                                    | 7       | Brier score + ECE                                                 |
| <code>icarm_thresholds()</code><br><code>icarm_plot_thresholds()</code>                                    | 7       | Full 0.10–0.90 threshold sweep                                    |
| <code>icarm_audit()</code>                                                                                 | 8       | Reproducible JSON audit trail                                     |
| <code>icarm_scorecard()</code>                                                                             | 8       | Full accountability scorecard                                     |

Table 4: Every `icarm` v0.1.0 function used in this workshop, and where.

## B Consolidated Code Booklet

This is the complete script, top to bottom, with the fill-in-the-blank exercises already completed.

Listing 5: Full consolidated script

```

1  ## =====
2  ## TEACHING THE MACHINE LEARNING STORY -- CONSOLIDATED CODE BOOKLET
3  ## Package: icarm v0.1.0 | Data: student-mat.csv (395 students, seed 2025)
4  ## =====
5
6  library(icarm); library(dplyr); library(ggplot2)
7
8  students <- read.csv2("student-mat.csv", stringsAsFactors = TRUE)
9  students$pass <- factor(ifelse(students$G3 >= 10, "Pass", "Fail"))
10 students$tier <- cut(students$G3, breaks = c(-1, 9, 14, 20),
11                      labels = c("Low", "Mid", "High"))
12
13 ## ---- Stage 3: Split ----
14 sp <- icarm_split(students, prop = 0.75, seed = 2025, stratify = "pass")
15 train_bin <- sp$train %>% select(-G1, -G2, -G3, -tier)
16 test_bin <- sp$test %>% select(-G1, -G2, -G3, -tier)
17
18 ## ---- Stage 4: Train ----
19 m_cart <- icarm_fit(pass ~ ., train_bin, model = "cart", positive = "
    Pass")
20 m_logit <- icarm_fit(pass ~ ., train_bin, model = "logistic", positive = "
    Pass")
21 m_l1 <- icarm_fit(pass ~ ., train_bin, model = "logistic_l1", positive = "
    Pass")
22 m_rf <- icarm_fit(pass ~ ., train_bin, model = "random_forest", positive = "
    Pass")
23 m_svm <- icarm_fit(pass ~ ., train_bin, model = "svm", positive = "
    Pass")
24
25 ## ---- Stage 5: Evaluate ----
26 pred_cart <- predict(m_cart, test_bin, type = "class")
27 icarm_metrics(test_bin$pass, pred_cart, positive = "Pass")
28
29 cmp <- icarm_compare(
30   list(CART = m_cart, Logistic = m_logit, 'Logistic (L1)' = m_l1,
31         RandomForest = m_rf, SVM = m_svm),
32   test_data = test_bin, outcome = "pass", positive = "Pass")
33 print(cmp)
34
35 ## ---- Stage 6: Explain ----
36 ex_rf <- icarm_explain(m_rf, data = test_bin, label = "Random Forest")
37 ex_cart <- icarm_explain(m_cart, data = test_bin, label = "CART")
38 icarm_plot_importance(ex_rf, n_features = 12)
39 icarm_plot_importance(ex_cart, n_features = 12)
40
41 case_a_id <- which(test_bin$pass == "Pass" & pred_cart == "Pass")[1]
42 case_b_id <- 28
43 case_c_id <- which(pred_cart == "Fail" & test_bin$pass == "Pass")[1]
44 predict(m_cart, test_bin[c(case_a_id, case_b_id, case_c_id), ], type = "prob")
45
46 ## ---- Stage 7: Audit ----
47 fair_sex <- icarm_fairness(m_rf, test_bin, outcome = "pass",
48                           protected = "sex", positive = "Pass")
49 fair_addr <- icarm_fairness(m_rf, test_bin, outcome = "pass",
50                             protected = "address", positive = "Pass")
51 print(fair_sex); print(fair_addr)
52

```

```

53 probs <- predict(m_rf, test_bin, type = "prob")[, "Pass"]
54 cal <- icarm_calibrate(m_rf, test_bin, outcome = "pass", positive = "Pass")
55 thr <- icarm_thresholds(test_bin$pass, probs, positive = "Pass")
56
57 ## ---- Stage 8: Decide -----
58 metrics_rf <- icarm_metrics(test_bin$pass, predict(m_rf, test_bin, type = "class"
59   ),
60   y_prob = probs, positive = "Pass")
61 icarm_audit(m_rf, metrics = metrics_rf, fairness = fair_sex,
62   analyst = "Your Name", path = "rf_audit_trail.json")
63 icarm_scorecard(m_rf, test_bin, outcome = "pass", protected = "sex",
64   positive = "Pass", analyst = "Your Name",
65   project = "ICOTS Workshop", path = "rf_scorecard.json")
66
67 ## ---- Capstone template -----
68 ATTR <- "Pstatus" # change per pair: Pstatus / internet / schoolsup / guardian
69 team_fair <- icarm_fairness(m_rf, test_bin, outcome = "pass",
70   protected = ATTR, positive = "Pass")
71 print(team_fair)
72
73 ## ---- Bonus: Cold Case Files (same data, two different questions) -----
74 train_reg <- sp$train %>% select(-G1, -G2, -pass, -tier)
75 test_reg <- sp$test %>% select(-G1, -G2, -pass, -tier)
76 m_lin <- icarm_fit(G3 ~ ., train_reg, model = "linear")
77 icarm_metrics(test_reg$G3, predict(m_lin, test_reg), type = "regression")
78
79 train_tier <- sp$train %>% select(-G1, -G2, -G3, -pass)
80 test_tier <- sp$test %>% select(-G1, -G2, -G3, -pass)
81 m_multi <- icarm_fit(tier ~ ., train_tier, model = "multinomial")
82 icarm_metrics(test_tier$tier, predict(m_multi, test_tier, type = "class"),
83   type = "multiclass")

```

## C References and Further Reading

---

- Cortez, P., & Silva, A. (2008). Using data mining to predict secondary school student performance. *Proceedings of 5th FUBUTEC Conference*, 5–12.
- d'Alessandro, B., O'Neil, C., & LaGatta, T. (2017). Conscientious classification: A data scientist's guide to discrimination-aware classification. *Big Data*, 5(2).
- Mitchell, S., Potash, E., Barocas, S., D'Amour, A., & Lum, K. (2021). Algorithmic fairness: Choices, assumptions, and definitions. *Annual Review of Statistics and Its Application*.
- `icarm` package documentation (CRAN, v0.1.0) — every function in Appendix A is documented there in full.

*Ask. Data. Split. Train. Evaluate. Explain. Audit. Decide.*

**Claim. Context. Case. Caveat.**